

E-commerce Data Analysis

Data source:

https://www.kaggle.com/datasetsolistbr/brazilian-e-commerceutm_source=chatgpt.com

Objectives

- Importing data
- Cleaning data
- Visualizing data

0.1 Data import

```
[24]: # import modules
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
[26]: # creation function to load data
def load(path):
    dataframe = pd.read_csv(path)
    return dataframe
# call function
df_customer = load ("/home/djodj/Downloads/archive/
→olist_customers_dataset.csv")
df_geo = load("/home/djodj/Downloads/archive/olist_geolocation_dataset.
→csv")
df_order_it = load("/home/djodj/Downloads/archive/
→olist_order_items_dataset.csv")
df_order_pay = load("/home/djodj/Downloads/archive/
→olist_order_payments_dataset.csv")
df_order_rev = load("/home/djodj/Downloads/archive/
→olist_order_reviews_dataset.csv")
```

```
df_order_dataset = load("/home/djodj/Downloads/archive/
↳olist_orders_dataset.csv")
df_product = load("/home/djodj/Downloads/archive/olist_products_dataset.
↳csv")
df_sellers = load("/home/djodj/Downloads/archive/olist_sellers_dataset.
↳csv")
df_categorie = load("/home/djodj/Downloads/archive/
↳product_category_name_translation.csv")
```

0.1.1 General information

Let's visualize each dataframe and its data type

```
[27]: df_customer.head()
```

```
[27]:
```

	customer_id	customer_unique_id	\
0	06b8999e2fba1a1fbc88172c00ba8bc7	861eff4711a542e4b93843c6dd7febb0	
1	18955e83d337fd6b2def6b18a428ac77	290c77bc529b7ac935b93aa66c333dc3	
2	4e7b3e00288586ebd08712fdd0374a03	060e732b5b29e8181a18229c7b0b2b5e	
3	b2b6027bc5c5109e529d4dc6358b12c3	259dac757896d24d7702b9acbbff3f3c	
4	4f2d8ab171c80ec8364f7c12e35b23ad	345ecd01c38d18a9036ed96c73b8d066	

	customer_zip_code_prefix	customer_city	customer_state
0	14409	franca	SP
1	9790	sao bernardo do campo	SP
2	1151	sao paulo	SP
3	8775	mogi das cruzeiras	SP
4	13056	campinas	SP

```
[5]: df_customer.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 99441 entries, 0 to 99440
Data columns (total 5 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   customer_id                          99441 non-null  object
1   customer_unique_id                   99441 non-null  object
2   customer_zip_code_prefix             99441 non-null  int64
3   customer_city                        99441 non-null  object
4   customer_state                       99441 non-null  object
dtypes: int64(1), object(4)
memory usage: 3.8+ MB
```

```
[28]: df_geo.head()
```

```
[28]: geolocation_zip_code_prefix geolocation_lat geolocation_lng \
0      1037      -23.545621      -46.639292
1      1046      -23.546081      -46.644820
2      1046      -23.546129      -46.642951
3      1041      -23.544392      -46.639499
4      1035      -23.541578      -46.641607

geolocation_city geolocation_state
0      sao paulo      SP
1      sao paulo      SP
2      sao paulo      SP
3      sao paulo      SP
4      sao paulo      SP
```

```
[6]: df_geo.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000163 entries, 0 to 1000162
Data columns (total 5 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   geolocation_zip_code_prefix          1000163 non-null int64
1   geolocation_lat                      1000163 non-null float64
2   geolocation_lng                      1000163 non-null float64
3   geolocation_city                     1000163 non-null object
4   geolocation_state                    1000163 non-null object
dtypes: float64(2), int64(1), object(2)
memory usage: 38.2+ MB
```

```
[7]: df_order_it.head()
```

```
[7]: order_id order_item_id \
0  00010242fe8c5a6d1ba2dd792cb16214      1
1  00018f77f2f0320c557190d7a144bdd3      1
2  000229ec398224ef6ca0657da4fc703e      1
3  00024acbcd0a6daa1e931b038114c75      1
4  00042b26cf59d7ce69dfabb4e55b4fd9      1

product_id seller_id \
0  4244733e06e7ecb4970a6e2683c13e61  48436dade18ac8b2bce089ec2a041202
1  e5f2d52b802189ee658865ca93d83a8f  dd7ddc04e1b6c2c614352b383efe2d36
2  c777355d18b72b67abbeef9df44fd0fd  5b51032eddd242adc84c38acab88f23d
3  7634da152a4610f1595efa32f14722fc  9d7a1d34a5052409006425275ba1c2b4
4  ac6c3623068f30de03045865e4e10089  df560393f3a51e74553ab94004ba5c87
```

	shipping_limit_date	price	freight_value
0	2017-09-19 09:45:35	58.90	13.29
1	2017-05-03 11:05:13	239.90	19.93
2	2018-01-18 14:48:30	199.00	17.87
3	2018-08-15 10:10:18	12.99	12.79
4	2017-02-13 13:57:51	199.90	18.14

```
[8]: df_order_it.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 112650 entries, 0 to 112649
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype
---  -
0   order_id              112650 non-null object
1   order_item_id         112650 non-null int64
2   product_id            112650 non-null object
3   seller_id             112650 non-null object
4   shipping_limit_date   112650 non-null object
5   price                 112650 non-null float64
6   freight_value         112650 non-null float64
dtypes: float64(2), int64(1), object(4)
memory usage: 6.0+ MB
```

```
[9]: df_order_pay.head()
```

```
[9]:
```

	order_id	payment_sequential	payment_type	\
0	b81ef226f3fe1789b1e8b2acac839d17	1	credit_card	
1	a9810da82917af2d9aefd1278f1dcfa0	1	credit_card	
2	25e8ea4e93396b6fa0d3dd708e76c1bd	1	credit_card	
3	ba78997921bbcdc1373bb41e913ab953	1	credit_card	
4	42fdf880ba16b47b59251dd489d4441a	1	credit_card	

	payment_installments	payment_value
0	8	99.33
1	1	24.39
2	1	65.71
3	8	107.78
4	2	128.45

```
[10]: df_order_pay.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 103886 entries, 0 to 103885
Data columns (total 5 columns):
#   Column                Non-Null Count  Dtype
---  -
#   Column                Non-Null Count  Dtype
```

```

---  -----
0   order_id          103886 non-null  object
1   payment_sequential 103886 non-null  int64
2   payment_type       103886 non-null  object
3   payment_installments 103886 non-null  int64
4   payment_value      103886 non-null  float64
dtypes: float64(1), int64(2), object(2)
memory usage: 4.0+ MB

```

```
[11]: df_order_rev.head()
```

```

[11]:
      review_id          order_id \
0  7bc2406110b926393aa56f80a40eba40  73fc7af87114b39712e6da79b0a377eb
1  80e641a11e56f04c1ad469d5645fdfde  a548910a1c6147796b98fdf73dbeba33
2  228ce5500dc1d8e020d8d1322874b6f0  f9e4b658b201a9f2ecdecbb34bed034b
3  e64fb393e7b32834bb789ff8bb30750e  658677c97b385a9be170737859d3511b
4  f7c4243c7fe1938f181bec41a392bdeb  8e6bfb81e283fa7e4f11123a3fb894f1

      review_score review_comment_title \
0                4                  NaN
1                5                  NaN
2                5                  NaN
3                5                  NaN
4                5                  NaN

      review_comment_message review_creation_date
→ \
0                NaN  2018-01-18 00:00:00
1                NaN  2018-03-10 00:00:00
2                NaN  2018-02-17 00:00:00
3      Recebi bem antes do prazo estipulado.  2017-04-21 00:00:00
4  Parabéns lojas lannister adorei comprar pela I...  2018-03-01 00:00:00

      review_answer_timestamp
0      2018-01-18 21:46:59
1      2018-03-11 03:05:13
2      2018-02-18 14:36:24
3      2017-04-21 22:02:06
4      2018-03-02 10:26:53

```

```
[12]: df_order_rev.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 99224 entries, 0 to 99223
Data columns (total 7 columns):

```

```

#      Column                                Non-Null Count  Dtype
---  -
0      review_id                            99224 non-null     object
1      order_id                             99224 non-null     object
2      review_score                         99224 non-null     int64
3      review_comment_title                 11568 non-null     object
4      review_comment_message               40977 non-null     object
5      review_creation_date                  99224 non-null     object
6      review_answer_timestamp               99224 non-null     object
dtypes: int64(1), object(6)
memory usage: 5.3+ MB

```

```
[13]: df_order_dataset.head()
```

```

[13]:
      order_id                                customer_id \
0  e481f51cbdc54678b7cc49136f2d6af7  9ef432eb6251297304e76186b10a928d
1  53cdb2fc8bc7dce0b6741e2150273451  b0830fb4747a6c6d20dea0b8c802d7ef
2  47770eb9100c2d0c44946d9cf07ec65d  41ce2a54c0b03bf3443c3d931a367089
3  949d5b44dbf5de918fe9c16f97b45f8a  f88197465ea7920adcdbec7375364d82
4  ad21c59c0840e6cb83a9ceb5573f8159  8ab97904e6daea8866dbdbc4fb7aad2c

      order_status  order_purchase_timestamp  order_approved_at \
0      delivered      2017-10-02 10:56:33  2017-10-02 11:07:15
1      delivered      2018-07-24 20:41:37  2018-07-26 03:24:27
2      delivered      2018-08-08 08:38:49  2018-08-08 08:55:23
3      delivered      2017-11-18 19:28:06  2017-11-18 19:45:59
4      delivered      2018-02-13 21:18:39  2018-02-13 22:20:29

      order_delivered_carrier_date  order_delivered_customer_date \
0      2017-10-04 19:55:00      2017-10-10 21:25:13
1      2018-07-26 14:31:00      2018-08-07 15:27:45
2      2018-08-08 13:50:00      2018-08-17 18:06:29
3      2017-11-22 13:39:59      2017-12-02 00:28:42
4      2018-02-14 19:46:34      2018-02-16 18:17:02

      order_estimated_delivery_date
0      2017-10-18 00:00:00
1      2018-08-13 00:00:00
2      2018-09-04 00:00:00
3      2017-12-15 00:00:00
4      2018-02-26 00:00:00

```

```
[14]: df_order_dataset.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 99441 entries, 0 to 99440

Data columns (total 8 columns):

#	Column	Non-Null Count	Dtype
0	order_id	99441 non-null	object
1	customer_id	99441 non-null	object
2	order_status	99441 non-null	object
3	order_purchase_timestamp	99441 non-null	object
4	order_approved_at	99281 non-null	object
5	order_delivered_carrier_date	97658 non-null	object
6	order_delivered_customer_date	96476 non-null	object
7	order_estimated_delivery_date	99441 non-null	object

dtypes: object(8)

memory usage: 6.1+ MB

```
[15]: df_product.head()
```

```
[15]:
```

	product_id	product_category_name	\
0	1e9e8ef04dbcff4541ed26657ea517e5	perfumaria	
1	3aa071139cb16b67ca9e5dea641aaa2f	artes	
2	96bd76ec8810374ed1b65e291975717f	esporte_lazer	
3	cef67bcfe19066a932b7673e239eb23d	bebes	
4	9dc1a7de274444849c219cff195d0b71	utilidades_domesticas	

	product_name_lenght	product_description_lenght	product_photos_qty	\
0	40.0	287.0	1.0	
1	44.0	276.0	1.0	
2	46.0	250.0	1.0	
3	27.0	261.0	1.0	
4	37.0	402.0	4.0	

	product_weight_g	product_length_cm	product_height_cm	product_width_cm
0	225.0	16.0	10.0	14.0
1	1000.0	30.0	18.0	20.0
2	154.0	18.0	9.0	15.0
3	371.0	26.0	4.0	26.0
4	625.0	20.0	17.0	13.0

```
[16]: df_product.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 32951 entries, 0 to 32950
Data columns (total 9 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   product_id                           32951 non-null  object
1   product_category_name                 32341 non-null  object
2   product_name_lenght                   32341 non-null  float64
3   product_description_lenght            32341 non-null  float64
4   product_photos_qty                   32341 non-null  float64
5   product_weight_g                      32949 non-null  float64
6   product_length_cm                     32949 non-null  float64
7   product_height_cm                     32949 non-null  float64
8   product_width_cm                      32949 non-null  float64
dtypes: float64(7), object(2)
memory usage: 2.3+ MB
```

```
[17]: df_sellers.head()
```

```
[17]:
```

	seller_id	seller_zip_code_prefix	\
0	3442f8959a84dea7ee197c632cb2df15	13023	
1	d1b65fc7debc3361ea86b5f14c68d2e2	13844	
2	ce3ad9de960102d0677a81f5d0bb7b2d	20031	
3	c0f3eea2e14555b6faeea3dd58c1b1c3	4195	
4	51a04a8a6bdbcb23deccc82b0b80742cf	12914	

	seller_city	seller_state
0	campinas	SP
1	mogi guacu	SP
2	rio de janeiro	RJ
3	sao paulo	SP
4	braganca paulista	SP

```
[18]: df_sellers.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3095 entries, 0 to 3094
Data columns (total 4 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   seller_id                           3095 non-null  object
1   seller_zip_code_prefix              3095 non-null  int64
2   seller_city                         3095 non-null  object
3   seller_state                        3095 non-null  object
```



```
dtypes: int64(1), object(3)
memory usage: 96.8+ KB
```

```
[19]: df_categorie.head()
```

```
[19]:      product_category_name product_category_name_english
0          beleza_saude          health_beauty
1  informatica_acessorios      computers_accessories
2          automotivo          auto
3      cama_mesa_banho      bed_bath_table
4      moveis_decoracao      furniture_decor
```

```
[20]: df_categorie.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 71 entries, 0 to 70
Data columns (total 2 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   product_category_name                 71 non-null    object
1   product_category_name_english         71 non-null    object
dtypes: object(2)
memory usage: 1.2+ KB
```

0.2 Data cleaning

The first step here is to identify the missing value in each dataframe, then manage that data, and finally convert the data types to the appropriate format.

0.2.1 Identifying missing value for each dataframe

```
[21]: df_customer.isnull().sum()
```

```
[21]: customer_id                0
      customer_unique_id       0
      customer_zip_code_prefix  0
      customer_city             0
      customer_state            0
      dtype: int64
```

For this dataframe there are no missing value

```
[22]: df_geo.isnull().sum()
```

```
[22]: geolocation_zip_code_prefix    0
      geolocation_lat                0
      geolocation_lng                0
      geolocation_city                0
      geolocation_state              0
      dtype: int64
```

No missing value is present for this dataframe

```
[23]: df_order_it.isnull().sum()
```

```
[23]: order_id                0
      order_item_id          0
      product_id             0
      seller_id              0
      shipping_limit_date     0
      price                  0
      freight_value          0
      dtype: int64
```

```
[26]: df_order_pay.isnull().sum()
```

```
[26]: order_id                0
      payment_sequential      0
      payment_type            0
      payment_installments    0
      payment_value           0
      dtype: int64
```

```
[24]: df_order_rev.isnull().sum()
```

```
[24]: review_id                0
      order_id                0
      review_score             0
      review_comment_title     87656
      review_comment_message   58247
      review_creation_date      0
      review_answer_timestamp   0
      dtype: int64
```

Note that this dataframe contains missing values.

```
[25]: df_order_dataset.isnull().sum()
```

```
[25]: order_id                0
      customer_id            0
```

```

order_status                0
order_purchase_timestamp    0
order_approved_at          160
order_delivered_carrier_date 1783
order_delivered_customer_date 2965
order_estimated_delivery_date 0
dtype: int64

```

```
[29]: df_product.isnull().sum()
```

```

[29]: product_id                0
      product_category_name    610
      product_name_lenght     610
      product_description_lenght 610
      product_photos_qty      610
      product_weight_g         2
      product_length_cm        2
      product_height_cm        2
      product_width_cm         2
      dtype: int64

```

```
[26]: df_sellers.isnull().sum()
```

```

[26]: seller_id                0
      seller_zip_code_prefix    0
      seller_city               0
      seller_state              0
      dtype: int64

```

```
[27]: df_categorie.isnull().sum()
```

```

[27]: product_category_name    0
      product_category_name_english 0
      dtype: int64

```

In conclusion, we note that the DataFrames `df_order_rev`, `df_order_dataset` and `df_product` contain missing values.

0.2.2 Handling Missing Data

`df_order_rev` - For the `order_rev` dataframe, let's remove the columns with missing values

```

[3]: df_order_rev.drop(["review_comment_title", "review_comment_message"],
    ↪axis=1, inplace=True)

```

df_order_dataset

- For the “df_order_dataset” dataframe, we impute missing values, noting that the ‘purchase’ date is generally the same as the “approved” date.

```
[4]: df_order_dataset['order_approved_at'] =_
      ↳df_order_dataset['order_approved_at'].
      ↳fillna(df_order_dataset["order_purchase_timestamp"])
```

- For the order_deliver column, we process it with the understanding that shipping takes place after approval, so we add 2 days. Similarly, for deliver, we add 5 days after delivery.

```
[5]: #conversion
df_order_dataset['order_delivered_carrier_date'] =_
      ↳df_order_dataset['order_delivered_carrier_date'].
      ↳astype("datetime64[ns]")
df_order_dataset['order_approved_at']_
      ↳=df_order_dataset['order_approved_at'].astype("datetime64[ns]")
# completement
df_order_dataset['order_delivered_carrier_date'] =_
      ↳df_order_dataset['order_delivered_carrier_date'].fillna(
      df_order_dataset['order_approved_at']+ pd.Timedelta(days=2))
```

```
[6]: df_order_dataset['order_delivered_customer_date'] =_
      ↳df_order_dataset['order_delivered_customer_date'].
      ↳astype("datetime64[ns]")
# complement
df_order_dataset['order_delivered_customer_date'] =_
      ↳df_order_dataset['order_delivered_customer_date'].fillna(
      df_order_dataset['order_delivered_carrier_date']+ pd.
      ↳Timedelta(days=5))
```

df_product

- Let’s remove the products which no name in the dataframe.

```
[7]: df_product.dropna(subset=["product_category_name"], axis=0, inplace=True)
```

```
[8]: df_product.dropna(subset=["product_weight_g"], axis=0, inplace=True)
```

0.2.3 data duplicated

```
[36]: df_customer.duplicated().sum()
```

```
[36]: np.int64(0)
```

```
[10]: df_geo.duplicated().sum()
```

```
[10]: np.int64(261831)
```

RLet's remove the duplicates from this dataframe

```
[9]: df_geo = df_geo.drop_duplicates()
```

```
[43]: df_order_it.duplicated().sum()
```

```
[43]: np.int64(0)
```

```
[44]: df_order_pay.duplicated().sum()
```

```
[44]: np.int64(0)
```

```
[45]: df_order_rev.duplicated().sum()
```

```
[45]: np.int64(0)
```

```
[46]: df_order_dataset.duplicated().sum()
```

```
[46]: np.int64(0)
```

```
[47]: df_product.duplicated().sum()
```

```
[47]: np.int64(0)
```

```
[49]: df_sellers.duplicated().sum()
```

```
[49]: np.int64(0)
```

```
[50]: df_categorie.duplicated().sum()
```

```
[50]: np.int64(0)
```

0.2.4 Data conversion

- The dataframes `customers`, `geo`, `order_pay`, `order_product`, and `categorie` are in the correct format.
- The DataFrames `df_order_it`, `df_order_rev` contain incorrect date formats; let's correct them.

```
[39]: df_order_it["shipping_limit_date"] = df_order_it["shipping_limit_date"].
      ↪astype("datetime64[ns]")
df_order_rev['review_answer_timestamp'] =
      ↪df_order_rev['review_answer_timestamp'].astype("datetime64[ns]")
df_order_rev['review_creation_date'] =
      ↪df_order_rev['review_creation_date'].astype("datetime64[ns]")
df_order_dataset['order_purchase_timestamp'] =
      ↪df_order_dataset['order_purchase_timestamp'].astype("datetime64[ns]")
df_order_dataset['order_estimated_delivery_date'] =
      ↪df_order_dataset['order_estimated_delivery_date'].
      ↪astype("datetime64[ns]")
```

0.2.5 Normalisation

- Let's normalize the product dataframe

```
[11]: df_product['product_weight_g'] = df_product['product_weight_g'] /
      ↪df_product['product_weight_g'].max()
df_product['product_length_cm'] = df_product['product_length_cm'] /
      ↪df_product['product_length_cm'].max()
df_product['product_height_cm'] = df_product['product_height_cm'] /
      ↪df_product['product_height_cm'].max()
df_product['product_width_cm'] = df_product['product_width_cm'] /
      ↪df_product['product_width_cm'].max()
```

0.2.6 Binning

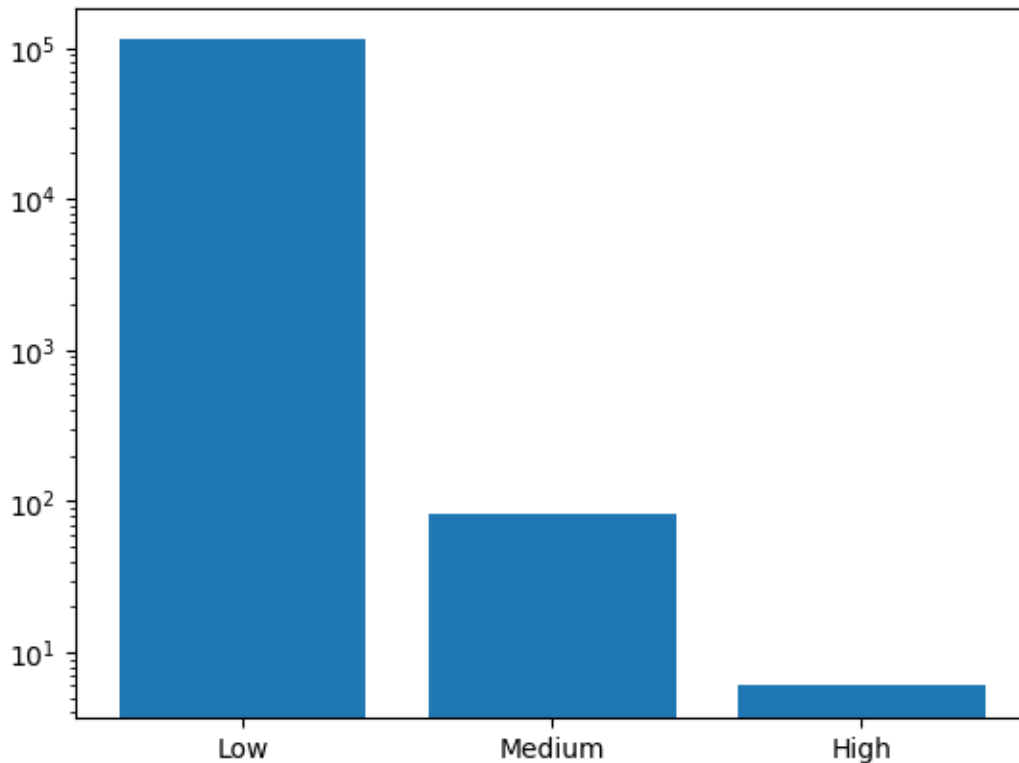
- Let's group the prices into three categories: low, medium, and high, and visualize the distribution using a histogram.

```
[14]: bins = np.linspace(min(df_order_it["price"]), max(df_order_it["price"]),
      ↪4)
group_names = ['Low', 'Medium', 'High']
df_order_it['price_binned'] = pd.cut(df_order_it['price'], bins,
      ↪labels=group_names, include_lowest=True )
```

```
[15]: df_order_it["price_binned"].value_counts()
```

```
[15]: price_binned
Low      112562
Medium    82
High       6
Name: count, dtype: int64
```

```
[16]: #from matplotlib import pyplot
#plt.hist(group_names, df_order_it["price_binned"].value_counts())
plt.bar(group_names, df_order_it["price_binned"].value_counts())
plt.yscale('log')
plt.show()
```



0.3 Exploratory data

let's view a quick summary of our dataframe using the `describe()` method.

```
[51]: df_order_it["price"].describe()
```

```
[51]: count    112650.000000
mean       120.653739
std        183.633928
min         0.850000
25%        39.900000
50%        74.990000
75%       134.900000
max       6735.000000
```

Name: price, dtype: float64

```
[53]: df_order_pay.describe()
```

```
[53]:      payment_sequential  payment_installments  payment_value
count      103886.000000      103886.000000      103886.000000
mean         1.092679         2.853349         154.100380
std          0.706584         2.687051         217.494064
min          1.000000         0.000000          0.000000
25%          1.000000         1.000000          56.790000
50%          1.000000         1.000000         100.000000
75%          1.000000         4.000000         171.837500
max          29.000000         24.000000        13664.080000
```

```
[56]: df_product.describe()
```

```
[56]:      product_name_lenght  product_description_lenght \
      ↳product_photos_qty \
count      32340.000000      32340.000000      32340.000000
mean         48.476592         771.492393          2.188961
std          10.245699         635.124831          1.736787
min           5.000000          4.000000          1.000000
25%          42.000000         339.000000          1.000000
50%          51.000000         595.000000          1.000000
75%          57.000000         972.000000          3.000000
max          76.000000        3992.000000         20.000000
```

```
      product_weight_g  product_length_cm  product_height_cm \
count      32340.000000      32340.000000      32340.000000
mean         0.056325         0.293853         16.958813
std          0.105858         0.161485         13.636115
min          0.000000         0.066667          2.000000
25%          0.007421         0.171429          8.000000
50%          0.017316         0.238095         13.000000
75%          0.047001         0.361905         21.000000
max          1.000000         1.000000        105.000000
```

```
      product_width_cm  product_height_cm
count      32340.000000      32340.000000
mean         0.196683         0.161513
std          0.102362         0.129868
min          0.050847         0.019048
25%          0.127119         0.076190
50%          0.169492         0.123810
```


75%	0.254237	0.200000
max	1.000000	1.000000